Exercícios de Python

Python puro, com problemas do mundo de segurança da informação

Como usar

São 20 exercícios em 5 blocos, em ordem crescente de dificuldade. Todos usam só a biblioteca padrão do Python (nada de `pip install`). O tema é de segurança, mas a prática é de Python: strings, listas, dicionários, sets, arquivos, regex, funções e classes. Tudo é defensivo e didático: validar, analisar, proteger.

Para cada exercício: leia o enunciado, escreva a função num arquivo `.py`, teste com os exemplos e só depois olhe o gabarito (últimas páginas). Se travar mais de 15 minutos, leia a dica. Se ainda travar, olhe o gabarito, feche, e reescreva de memória no dia seguinte.

Log de exemplo

Os exercícios do Bloco 3 em diante usam este log. Salve como `logs.txt` na mesma pasta do seu código. Depois, aumente o arquivo com mais linhas suas para testar melhor.

```
2026-11-12 08:01:15 INFO login_ok user=ana ip=192.168.0.10
2026-11-12 08:02:03 WARNING login_falhou user=admin ip=45.33.12.9
2026-11-12 08:02:05 WARNING login_falhou user=admin ip=45.33.12.9
2026-11-12 08:02:08 WARNING login_falhou user=root ip=45.33.12.9
2026-11-12 08:03:41 INFO login_ok user=bruno ip=192.168.0.22
2026-11-12 08:04:10 ERROR acesso_negado user=bruno ip=192.168.0.22
2026-11-12 08:05:30 WARNING login_falhou user=admin ip=203.0.113.7
2026-11-12 08:06:02 INFO logout user=ana ip=192.168.0.10
2026-11-12 08:07:19 WARNING login_falhou user=admin ip=45.33.12.9
2026-11-12 08:07:25 WARNING login_falhou user=admin ip=45.33.12.9
```

Sumário

#	Exercício	Nível
1	Medidor de força de senha	Fácil
2	Cifra de César	Fácil
3	Validador de IPv4	Fácil
4	Mascarar e-mail	Fácil
5	Falhas de login por usuário	Fácil
6	Detector de força bruta	Médio
7	Lista de bloqueio com sets	Fácil
8	Gerador de senha segura	Médio
9	Contar linhas por nível	Fácil
10	Extrair IPs com regex	Médio
11	Top IPs mais frequentes	Fácil
12	Relatório em CSV	Médio
13	Hash de texto e de arquivo	Fácil
14	Verificador de integridade	Fácil
15	Base64 e XOR	Médio
16	Por que PIN de 4 dígitos é fraco	Médio
17	Classe Usuario com bloqueio	Médio
18	Logger de eventos de segurança	Médio
19	Política de senha configurável	Difícil
20	Mini projeto: analisador de log	Difícil

Bloco 1 - Strings e lógica básica

Exercício 1: Medidor de força de senha

Nível: Fácil

Crie `forca_senha(senha)` que devolve uma pontuação de 0 a 5. Ganha 1 ponto para cada regra cumprida: tem 8 ou mais caracteres, tem letra minúscula, tem letra maiúscula, tem número, tem símbolo (qualquer caractere que não seja letra nem número).

```
forca_senha("abc")        -> 1
forca_senha("Abcdef12!")  -> 5
```

Dica: Use `any()` com um generator e os métodos `islower`, `isupper`, `isdigit`, `isalnum`.

Exercício 2: Cifra de César

Nível: Fácil

Crie `cesar(texto, chave)` que desloca cada letra `chave` posições no alfabeto. Mantenha maiúsculas e minúsculas e deixe espaços e símbolos como estão. Para decifrar, use a chave negativa.

```
cesar("Python Seguro", 3)   -> "Sbwkrq Vhjxur"
cesar("Sbwkrq Vhjxur", -3)  -> "Python Seguro"
```

Dica: `ord()` e `chr()` convertem letra em número e vice-versa. O operador `%` 26 faz o alfabeto dar a volta.

Exercício 3: Validador de IPv4

Nível: Fácil

Crie `ipv4_valido(ip)` que devolve `True` ou `False`. Um IPv4 válido tem 4 partes separadas por ponto, e cada parte é um número de 0 a 255. Não use a biblioteca `ipaddress`. Desafio extra: rejeite partes com zero à esquerda, como `01`.

```
ipv4_valido("192.168.0.1")  -> True
ipv4_valido("256.1.1.1")    -> False
ipv4_valido("1.2.3")        -> False
ipv4_valido("a.b.c.d")      -> False
```

Dica: `split(".")`, `isdigit()` e uma comparação encadeada: `0 <= n <= 255`.

Exercício 4: Mascarar e-mail

Nível: Fácil

Crie `mascarar_email(email)` que mantém só a primeira letra do usuário e troca o resto por `*`. O domínio fica intacto.

```
mascarar_email("carlos@empresa.com")  -> "c*****@empresa.com"
```

Dica: `split("@")` e multiplicação de string: `"*" * 5`.

Bloco 2 - Listas, dicionários e sets

Exercício 5: Falhas de login por usuário

Nível: Fácil

Você recebe uma lista de tuplas `(usuario, status)`, onde status é `"ok"` ou `"falha"`. Crie `falhas_por_usuario(tentativas)` que devolve um dicionário com a quantidade de falhas de cada usuário. Quem não falhou não aparece.

```
tentativas = [("ana","ok"), ("admin","falha"), ("admin","falha"),
              ("bruno","ok"), ("admin","falha")]
falhas_por_usuario(tentativas)  -> {"admin": 3}
```

Dica: `dicionario.get(chave, 0) + 1` evita erro quando a chave ainda não existe.

Exercício 6: Detector de força bruta

Nível: Médio

Crie `suspeitos(ips_com_falha, limite=5)`. Recebe uma lista com um IP para cada falha de login e devolve a lista de IPs que falharam `limite` vezes ou mais. Faça primeiro com dicionário e depois com `collections.Counter`.

```
ips = ["1.1.1.1"]*5 + ["2.2.2.2"]*2
suspeitos(ips)            -> ["1.1.1.1"]
suspeitos(ips, limite=2)  -> ["1.1.1.1", "2.2.2.2"]
```

Dica: `Counter(lista)` já conta tudo. `.items()` devolve pares (ip, quantidade).

Exercício 7: Lista de bloqueio com sets

Nível: Fácil

Crie `bloqueados_que_acessaram(acessos, bloqueados)`. `acessos` é uma lista de IPs (com repetição) e `bloqueados` é uma lista de IPs proibidos. Devolva um set com os IPs bloqueados que apareceram nos acessos.

```
acessos = ["192.168.0.10", "45.33.12.9", "192.168.0.22", "45.33.12.9"]
bloqueados = ["45.33.12.9", "203.0.113.7"]
bloqueados_que_acessaram(acessos, bloqueados)  -> {"45.33.12.9"}
```

Dica: Operador `&` entre dois sets faz a interseção.

Exercício 8: Gerador de senha segura

Nível: Médio

Crie `gerar_senha(tamanho=16)` usando o módulo `secrets` (e não `random`). A senha deve ter ao menos uma minúscula, uma maiúscula, um número e um símbolo. Se a senha sorteada não cumprir, sorteie outra. Depois responda por escrito: por que `random` não serve para isso?

```
gerar_senha()    -> "k9#Tq2Lm!xW7pRzB"   (muda a cada chamada)
gerar_senha(24)  -> senha com 24 caracteres
```

Dica: `secrets.choice(alfabeto)` e `string.ascii_letters`, `string.digits`.

Bloco 3 - Arquivos, logs e regex

Exercício 9: Contar linhas por nível

Nível: Fácil

Salve o log de exemplo (página 2) como `logs.txt`. Crie `contar_niveis(caminho)` que lê o arquivo e devolve um dicionário com a quantidade de linhas de cada nível (`INFO`, `WARNING`, `ERROR`). O nível é a 3ª palavra de cada linha.

```
contar_niveis("logs.txt")  -> {"INFO": 3, "WARNING": 6, "ERROR": 1}
```

Dica: `with open(...) as f:` e `linha.split()`.

Exercício 10: Extrair IPs com regex

Nível: Médio

Crie `extrair_ips(caminho)` que devolve uma lista com todos os IPs do log, na ordem em que aparecem (com repetição). Use o módulo `re`. O IP aparece depois de `ip=`.

```
extrair_ips("logs.txt")[:3]  -> ["192.168.0.10", "45.33.12.9", "45.33.12.9"]
```

Dica: Padrão: `r"ip=(\d{1,3}(?:\.\d{1,3}){3})"` e `padrao.findall(linha)`.

Exercício 11: Top IPs mais frequentes

Nível: Fácil

Crie `top_ips(caminho, n=5)` que devolve os `n` IPs mais frequentes do log, como lista de pares `(ip, quantidade)`. Reaproveite `extrair_ips`.

```
top_ips("logs.txt", 2)  -> [("45.33.12.9", 5), ("192.168.0.10", 2)]
```

Dica: `Counter(...).most_common(n)`.

Exercício 12: Relatório em CSV

Nível: Médio

Crie `salvar_relatorio(pares, saida="relatorio.csv")` que grava em CSV a lista de pares devolvida por `top_ips`. A primeira linha é o cabeçalho `ip,ocorrencias`. Use o módulo `csv`.

```
salvar_relatorio(top_ips("logs.txt", 2))

relatorio.csv:
ip,ocorrencias
45.33.12.9,5
192.168.0.10,2
```

Dica: Abra com `"w", newline=""` e use `csv.writer(f).writerow([...])`.

Bloco 4 - Hash, Base64 e criptografia simples

Exercício 13: Hash de texto e de arquivo

Nível: Fácil

Crie `hash_texto(texto)` e `hash_arquivo(caminho)`, ambas devolvendo o SHA-256 em hexadecimal. No arquivo, leia em blocos de 8192 bytes, para funcionar mesmo com arquivos grandes.

```
hash_texto("abc")
-> "ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad"
```

Dica: `hashlib.sha256()`, `.update(bytes)`, `.hexdigest()`. Texto precisa de `.encode("utf-8")`.

Exercício 14: Verificador de integridade

Nível: Fácil

Crie `verificar_integridade(caminho, hash_esperado)` que devolve `True` se o hash do arquivo for igual ao esperado. Teste com um arquivo, depois mude uma letra dentro dele e veja o resultado virar `False`. É assim que se confere um download.

```
verificar_integridade("logs.txt", hash_original)  -> True
(depois de editar o arquivo)                      -> False
```

Dica: Reaproveite `hash_arquivo`. Use `.lower()` para ignorar diferença de maiúscula no hash.

Exercício 15: Base64 e XOR

Nível: Médio

Crie `b64_codificar(texto)`, `b64_decodificar(b64)` e `xor_bytes(dados, chave)`. As duas primeiras trabalham com `str`. `xor_bytes` recebe e devolve `bytes`, repetindo a chave quando ela é menor que os dados. Verifique que aplicar o XOR duas vezes com a mesma chave devolve o original. Depois escreva: Base64 e XOR protegem alguma coisa de verdade?

```
b64_codificar("segredo")   -> "c2VncmVkbw=="
b64_decodificar("c2VncmVkbw==")  -> "segredo"
xor_bytes(xor_bytes(b"ola", b"k"), b"k")  -> b"ola"
```

Dica: Módulo `base64`: `b64encode` e `b64decode` trabalham com bytes. No XOR, `chave[i % len(chave)]`.

Exercício 16: Por que PIN de 4 dígitos é fraco

Nível: Médio

Gere o SHA-256 de um PIN seu de 4 dígitos. Depois crie `descobrir_pin(hash_alvo)` que testa todos os PINs de `0000` a `9999` até achar o que gera aquele hash. Meça o tempo com `time.perf_counter()`. Conclusão a escrever: quantas tentativas no pior caso e o que isso diz sobre senhas curtas.

```
alvo = hash_texto("4821")
descobrir_pin(alvo)  -> "4821"
```

Dica: `f"{n:04d}"` formata o número com zeros à esquerda.

Bloco 5 - Classes e mini projeto

Exercício 17: Classe Usuario com bloqueio

Nível: Médio

Crie a classe `Usuario(nome, senha)`. Ela guarda apenas o hash da senha, nunca a senha. O método `login(senha)` devolve `True` ou `False`. Depois de 3 falhas seguidas a conta fica bloqueada, e login com conta bloqueada devolve `False` mesmo com a senha certa. Login correto zera o contador de falhas.

```
u = Usuario("ana", "S3nha!")
u.login("x"); u.login("y"); u.login("z")
u.bloqueado          -> True
u.login("S3nha!")    -> False
```

Dica: Atributos `self.falhas` e `self.bloqueado`. Constante de classe `MAX_TENTATIVAS = 3`.

Exercício 18: Logger de eventos de segurança

Nível: Médio

Crie a classe `LoggerSeguranca(arquivo)` com o método `registrar(nivel, evento, **dados)`. Ele acrescenta ao arquivo uma linha no mesmo formato do log de exemplo. Se o nível não for `INFO`, `WARNING` ou `ERROR`, levante `ValueError`.

```
log = LoggerSeguranca("novo.log")
log.registrar("WARNING", "login_falhou", user="admin", ip="1.2.3.4")

novo.log:
2026-11-12 09:15:02 WARNING login_falhou user=admin ip=1.2.3.4
```

Dica: `datetime.now()` com f-string `f"{agora:%Y-%m-%d %H:%M:%S}"`. Abra o arquivo com `"a"` (append).

Exercício 19: Política de senha configurável

Nível: Difícil

Crie a classe `PoliticaSenha(tamanho_min=8, exigir_maiuscula=True, exigir_numero=True, exigir_simbolo=True)`. O método `validar(senha)` devolve uma lista de erros em texto. Lista vazia significa senha aprovada. Rejeite também senhas de uma lista de senhas comuns (`123456`, `password`, `senha123`, `admin`), sem diferenciar maiúscula de minúscula.

```
p = PoliticaSenha(tamanho_min=10)
p.validar("Abc1!")
-> ["mínimo de 10 caracteres"]
p.validar("Senha-Forte-2026")  -> []
```

Dica: Monte a lista `erros = []` e use `append` em cada regra que falhar.

Exercício 20: Mini projeto: analisador de log

Nível: Difícil

Monte um script que roda no terminal: `python analisador.py logs.txt --limite 5`. Ele deve mostrar (1) quantas linhas há por nível, (2) os 3 IPs com mais falhas de login (linhas com `login_falhou`), (3) um alerta para cada IP com falhas igual ou acima do limite. Use `argparse`, `re`, `Counter` e separe a lógica em funções.

```
Linhas por nivel: {'INFO': 3, 'WARNING': 6, 'ERROR': 1}
Top falhas por IP: [('45.33.12.9', 5), ('203.0.113.7', 1)]
ALERTA: 45.33.12.9 com 5 falhas
```

Dica: Uma função `analisar(caminho, limite)` que devolve os resultados, e uma `main()` que só cuida de argumentos e de imprimir. Termine com `if __name__ == "__main__":`.

Gabarito

Tente resolver antes de olhar. Existe mais de um jeito certo de resolver; o que importa é passar nos exemplos.

Exercício 1: Medidor de força de senha

```
def forca_senha(senha):
    pontos = 0
    if len(senha) >= 8:
        pontos += 1
    if any(c.islower() for c in senha):
        pontos += 1
    if any(c.isupper() for c in senha):
        pontos += 1
    if any(c.isdigit() for c in senha):
        pontos += 1
    if any(not c.isalnum() for c in senha):
        pontos += 1
    return pontos
```

Exercício 2: Cifra de César

```
def cesar(texto, chave):
    resultado = ""
    for c in texto:
        if c.isalpha() and c.isascii():
            base = ord("A") if c.isupper() else ord("a")
            resultado += chr((ord(c) - base + chave) % 26 + base)
        else:
            resultado += c
    return resultado
```

Exercício 3: Validador de IPv4

```
def ipv4_valido(ip):
    partes = ip.split(".")
    if len(partes) != 4:
        return False
    for p in partes:
        if not p.isdigit() or not 0 <= int(p) <= 255:
            return False
    return True
```

Exercício 4: Mascarar e-mail

```
def mascarar_email(email):
    usuario, dominio = email.split("@")
    return usuario[0] + "*" * (len(usuario) - 1) + "@" + dominio
```

Exercício 5: Falhas de login por usuário

```
def falhas_por_usuario(tentativas):
    falhas = {}
    for usuario, status in tentativas:
        if status == "falha":
            falhas[usuario] = falhas.get(usuario, 0) + 1
    return falhas
```

Exercício 6: Detector de força bruta

```
from collections import Counter

def suspeitos(ips_com_falha, limite=5):
    contagem = Counter(ips_com_falha)
    return [ip for ip, n in contagem.items() if n >= limite]
```

Exercício 7: Lista de bloqueio com sets

```
def bloqueados_que_acessaram(acessos, bloqueados):
    return set(acessos) & set(bloqueados)
```

Exercício 8: Gerador de senha segura

```
import secrets
import string

SIMBOLOS = "!@#$%&*?"

def gerar_senha(tamanho=16):
    alfabeto = string.ascii_letters + string.digits + SIMBOLOS
    while True:
        senha = "".join(secrets.choice(alfabeto) for _ in range(tamanho))
        if (any(c.islower() for c in senha)
                and any(c.isupper() for c in senha)
                and any(c.isdigit() for c in senha)
                and any(c in SIMBOLOS for c in senha)):
            return senha
```

Exercício 9: Contar linhas por nível

```
def contar_niveis(caminho):
    contagem = {}
    with open(caminho, encoding="utf-8") as f:
        for linha in f:
            partes = linha.split()
            if len(partes) >= 3:
                nivel = partes[2]
                contagem[nivel] = contagem.get(nivel, 0) + 1
    return contagem
```

Exercício 10: Extrair IPs com regex

```
import re

PADRAO_IP = re.compile(r"ip=(\d{1,3}(?:\.\d{1,3}){3})")

def extrair_ips(caminho):
    ips = []
    with open(caminho, encoding="utf-8") as f:
        for linha in f:
            ips.extend(PADRAO_IP.findall(linha))
    return ips
```

Exercício 11: Top IPs mais frequentes

```
from collections import Counter

def top_ips(caminho, n=5):
    return Counter(extrair_ips(caminho)).most_common(n)
```

Exercício 12: Relatório em CSV

```
import csv

def salvar_relatorio(pares, saida="relatorio.csv"):
    with open(saida, "w", newline="", encoding="utf-8") as f:
        escritor = csv.writer(f)
        escritor.writerow(["ip", "ocorrencias"])
        for ip, n in pares:
            escritor.writerow([ip, n])
```

Exercício 13: Hash de texto e de arquivo

```
import hashlib

def hash_texto(texto):
    return hashlib.sha256(texto.encode("utf-8")).hexdigest()

def hash_arquivo(caminho):
    h = hashlib.sha256()
    with open(caminho, "rb") as f:
        for bloco in iter(lambda: f.read(8192), b""):
            h.update(bloco)
    return h.hexdigest()
```

Exercício 14: Verificador de integridade

```
def verificar_integridade(caminho, hash_esperado):
    return hash_arquivo(caminho) == hash_esperado.strip().lower()
```

Exercício 15: Base64 e XOR

```
import base64

def b64_codificar(texto):
    return base64.b64encode(texto.encode("utf-8")).decode("ascii")

def b64_decodificar(b64):
    return base64.b64decode(b64).decode("utf-8")

def xor_bytes(dados, chave):
    return bytes(b ^ chave[i % len(chave)] for i, b in enumerate(dados))
```

Exercício 16: Por que PIN de 4 dígitos é fraco

```
import hashlib

def descobrir_pin(hash_alvo):
    for n in range(10000):
        pin = f"{n:04d}"
        if hashlib.sha256(pin.encode()).hexdigest() == hash_alvo:
            return pin
    return None
```

Exercício 17: Classe Usuario com bloqueio

```
import hashlib

class Usuario:
    MAX_TENTATIVAS = 3

    def __init__(self, nome, senha):
        self.nome = nome
        self._hash = hashlib.sha256(senha.encode()).hexdigest()
        self.falhas = 0
        self.bloqueado = False

    def login(self, senha):
        if self.bloqueado:
            return False
        if hashlib.sha256(senha.encode()).hexdigest() == self._hash:
            self.falhas = 0
            return True
        self.falhas += 1
        if self.falhas >= self.MAX_TENTATIVAS:
            self.bloqueado = True
        return False
```

Exercício 18: Logger de eventos de segurança

```
from datetime import datetime

class LoggerSeguranca:
    NIVEIS = ("INFO", "WARNING", "ERROR")

    def __init__(self, arquivo):
        self.arquivo = arquivo

    def registrar(self, nivel, evento, **dados):
        if nivel not in self.NIVEIS:
            raise ValueError(f"nivel invalido: {nivel}")
        extras = " ".join(f"{k}={v}" for k, v in dados.items())
        agora = f"{datetime.now():%Y-%m-%d %H:%M:%S}"
        linha = f"{agora} {nivel} {evento} {extras}".strip()
        with open(self.arquivo, "a", encoding="utf-8") as f:
            f.write(linha + "\n")
```

Exercício 19: Política de senha configurável

```
class PoliticaSenha:
    COMUNS = {"123456", "password", "senha123", "admin"}

    def __init__(self, tamanho_min=8, exigir_maiuscula=True,
                 exigir_numero=True, exigir_simbolo=True):
        self.tamanho_min = tamanho_min
        self.exigir_maiuscula = exigir_maiuscula
        self.exigir_numero = exigir_numero
        self.exigir_simbolo = exigir_simbolo

    def validar(self, senha):
        erros = []
        if len(senha) < self.tamanho_min:
            erros.append(f"mínimo de {self.tamanho_min} caracteres")
        if self.exigir_maiuscula and not any(c.isupper() for c in senha):
            erros.append("falta letra maiúscula")
        if self.exigir_numero and not any(c.isdigit() for c in senha):
            erros.append("falta número")
        if self.exigir_simbolo and all(c.isalnum() for c in senha):
            erros.append("falta símbolo")
        if senha.lower() in self.COMUNS:
            erros.append("senha comum")
        return erros
```

Exercício 20: Mini projeto: analisador de log

```
import argparse
import re
from collections import Counter

PADRAO_IP = re.compile(r"ip=(\d{1,3}(?:\.\d{1,3}){3})")

def analisar(caminho, limite):
    niveis = Counter()
    falhas_por_ip = Counter()
    with open(caminho, encoding="utf-8") as f:
        for linha in f:
            partes = linha.split()
            if len(partes) < 3:
                continue
            niveis[partes[2]] += 1
            if "login_falhou" in linha:
                achado = PADRAO_IP.search(linha)
                if achado:
                    falhas_por_ip[achado.group(1)] += 1
    alertas = [ip for ip, n in falhas_por_ip.items() if n >= limite]
    return niveis, falhas_por_ip, alertas

def main():
    parser = argparse.ArgumentParser(description="Analisador de log")
    parser.add_argument("arquivo")
    parser.add_argument("--limite", type=int, default=5)
    args = parser.parse_args()
    niveis, falhas, alertas = analisar(args.arquivo, args.limite)
    print("Linhas por nivel:", dict(niveis))
    print("Top falhas por IP:", falhas.most_common(3))
    for ip in alertas:
        print(f"ALERTA: {ip} com {falhas[ip]} falhas")

if __name__ == "__main__":
    main()
```